ShelfEye ESP32-S3: Commands

Every command you need to set up, flash and check the ESP32-S3 camera board. Run them in a terminal on your computer (VS Code's terminal, PowerShell or Command Prompt), with the board plugged in over USB-C.

1. One-time setup (already done on this computer)

PlatformIO, the tool that builds and uploads the firmware, is already installed on this computer, together with everything it downloads for the ESP32 (about 2 GB). To check it:

```
python -m platformio --version
```

It should print `PlatformIO Core, version 6.2.0` or newer. Only on a **different** computer do you first need to run `pip install platformio`.

Use `python -m platformio` exactly as shown: the short `pio` command isn't set up on this computer.

2. Go to the ESP32 project folder

Every command after this one must be run from inside this folder:

```
cd C:\Users\12345\Desktop\Projects\Gogo_Plus\shelfeye\device\esp32s3
```

If you're already in the `Gogo_Plus` folder, this is enough:

```
cd shelfeye\device\esp32s3
```

3. Fill in your settings (once)

Open `shelfeye\device\esp32s3\data\.env` and make sure these are filled in:

- `WIFI_SSID` and `WIFI_PASSWORD`: your 2.4 GHz Wi-Fi
- `OPENAI_API_KEY`: your OpenAI key (the same one the Pi uses)
- `SHELFEYE_DEVICE_TOKEN`: the same value as on Render

This file is never uploaded to GitHub (it's in `.gitignore`).

4. Send the settings and firmware to the board

Send the settings

```
python -m platformio run -t uploadfs
```

Sends `data/.env` into the board's flash storage.

Build and install the firmware

```
python -m platformio run -t upload
```

Builds the firmware and installs it on the board. Takes about 1-2 minutes (the ESP32 compiler is already downloaded on this computer).

Watch the log

```
python -m platformio device monitor
```

Shows the board's log. Press **Ctrl + C** to stop watching (the board keeps running).

A good start looks like:

```
Storage: flash
Camera ready (sensor PID 0x3660)
Wi-Fi connected to your-wifi, IP 192.168.x.x
... ShelfEye shelf-2 ready: photo every 180 s, staying awake
... Counting: first photo since start-up
... Epilim Chrono 500: 0 -> 3 (+3)
```

5. Everyday commands

To...	Run (from the esp32s3 folder)
Change a setting (Wi-Fi, key, interval...)	edit `data\.env`, then `python -m platformio run -t uploadfs`
Install changed code	`python -m platformio run -t upload`
Watch the log	`python -m platformio device monitor`
Only check the code compiles (no board needed)	`python -m platformio run`
Restart the board	press its **RST** button, or unplug and replug USB-C

Changing a setting never needs `python -m platformio run -t upload`; the firmware reads `data/.env` from flash every time it starts.

6. If something goes wrong

Upload can't find the board / "Could not open port"

- Use a USB-C cable that carries data (some only charge).
- Close any other program using the port, including another `python -m platformio device monitor`.
- List the ports and pick the board's one:

```
python -m platformio device list
python -m platformio run -t upload --upload-port COM5
python -m platformio device monitor --port COM5
```

(replace `COM5` with the port shown for the board)

Upload starts but fails to connect

Hold the board's **BOOT** button, press and release **RST**, release **BOOT**, then run the upload command again.

The log shows nothing

In `platformio.ini`, change `-DARDUINO_USB_CDC_ON_BOOT=1` to `=0`, then run `python -m platformio run -t upload` again.

"No /.env found on the flash"

Run `python -m platformio run -t uploadfs` (the settings file must be at `data\.env`).

"Camera init failed"

The camera pins in `src\board_pins.h` don't match the board. Send a photo of the board's pinout sheet to get them fixed.

Start completely fresh (wipes the firmware and settings from the board)

```
python -m platformio run -t erase
python -m platformio run -t uploadfs
python -m platformio run -t upload
```